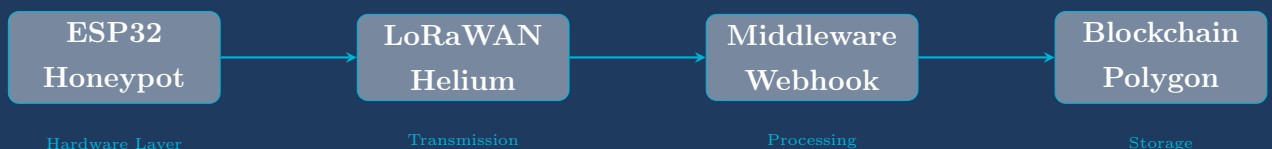

BLOCKCHAIN & IOT CYBERSECURITY PROJECT REPORT

Decentralized IoT Honeypot & Threat Registry

Sistem Deteksi Ancaman Berbasis LoRaWAN & Blockchain
dengan Immutable Smart Contract Threat Logging



Kelompok : Cybersecurity Research Team
Mata Kuliah : Blockchain Technology
Program : Teknik Komputer
Tahun : 2026

Contents

1	Pendahuluan	5
1.1	Latar Belakang	5
1.2	Rumusan Masalah	6
1.3	Tujuan Penelitian	6
1.4	Manfaat Penelitian	7
2	Tinjauan Pustaka & Landasan Teori	8
2.1	IoT Honeypot: Konsep dan Implementasi	8
2.1.1	Definisi Honeypot	8
2.1.2	Serangan Umum pada Perangkat IoT	9
2.2	ESP32: Platform Mikrokontroler untuk Honeypot	9
2.2.1	Spesifikasi Teknis	9
2.2.2	Kapabilitas Honeypot pada ESP32	10
2.3	LoRaWAN & Jaringan Helium	10
2.3.1	LoRa dan LoRaWAN	10
2.3.2	Jaringan Helium (The People's Network)	10
2.3.3	Relevansi untuk Sistem Keamanan	11
2.4	Blockchain & Smart Contract	11
2.4.1	Mengapa Blockchain untuk Threat Logging?	11
2.4.2	Ethereum Layer-2: Polygon & Arbitrum	12
2.4.3	Solidity: Bahasa Smart Contract	12
3	Tujuan dan Ruang Lingkup Proyek	13

3.1	Tujuan Utama	13
3.1.1	Tujuan 1: Implementasi Low-Interaction IoT Honeypot	13
3.1.2	Tujuan 2: Out-of-Band Transmission via Helium LoRaWAN	13
3.1.3	Tujuan 3: Immutable Blockchain Threat Registry	14
3.1.4	Tujuan 4: Dashboard Monitoring Terdesentralisasi	14
3.2	Ruang Lingkup Proyek	14
3.2.1	Dalam Ruang Lingkup	14
3.2.2	Di Luar Ruang Lingkup	15
4	Arsitektur Sistem & Alur Data	16
4.1	Arsitektur End-to-End	16
4.2	Alur Data Detail	16
4.2.1	Fase 1: Deteksi Serangan (ESP32)	16
4.2.2	Fase 2: Transmisi Out-of-Band (LoRa)	17
4.2.3	Fase 3: Middleware Processing	17
4.2.4	Fase 4: On-Chain Storage (Polygon)	17
5	Tools dan Teknologi yang Digunakan	19
5.1	Hardware Layer	19
5.1.1	ESP32 Development Board	19
5.1.2	Modul LoRa: RFM95W / SX1276	19
5.2	Network & Connectivity Layer	19
5.2.1	LoRaWAN Protocol Stack	19
5.2.2	Helium Console & LongFi Protocol	20
5.3	Blockchain Layer	21
5.3.1	Solidity	21
5.3.2	Polygon & Arbitrum (Ethereum L2)	21
5.3.3	Hardhat & Remix IDE	21
5.4	Middleware & Web3 Layer	22
5.4.1	Helium Console Webhooks	22

5.4.2	Ethers.js	22
5.5	Frontend Layer	22
5.5.1	Next.js	22
5.5.2	MetaMask Integration	23
6	Implementasi Teknis	24
6.1	Smart Contract: ThreatRegistry	24
6.1.1	Kode Kontrak Dasar	24
6.1.2	Versi Diperkuat dengan Access Control	25
6.2	Firmware ESP32 Honeypot	27
6.2.1	Struktur Utama Firmware	27
6.3	Middleware: Node.js Webhook Bridge	30
7	Daftar Referensi	32
7.1	Referensi Utama	32
7.1.1	Helium Network Documentation	32
7.1.2	Solidity & Ethereum Documentation	33
7.1.3	Konsep IoT Honeypot (IEEE/Jurnal Akademik)	33
7.2	Referensi Tambahan	33
7.2.1	Ethers.js Documentation	34
7.2.2	OWASP Internet of Things Project	34
7.2.3	OpenZeppelin Contracts	35
7.2.4	Referensi Tambahan Lainnya	35
8	Rencana Pengujian & Evaluasi	36
8.1	Skenario Pengujian	36
8.1.1	Test Case 1: Deteksi Port Scan	36
8.1.2	Test Case 2: Out-of-Band Failover	36
8.1.3	Test Case 3: Immutability Verification	37
8.2	Metrik Keberhasilan	37

9	Kesimpulan & Pengembangan Lanjutan	38
9.1	Kesimpulan	38
9.2	Pengembangan Lanjutan	39

Chapter 1

Pendahuluan

1.1 Latar Belakang

Ancaman siber terhadap perangkat *Internet of Things* (IoT) meningkat secara eksponensial seiring bertambahnya jumlah perangkat yang terhubung ke internet. Laporan Kaspersky Lab (2023) mencatat lebih dari **100 juta serangan** yang menargetkan perangkat IoT hanya dalam semester pertama 2023, meningkat 41% dibandingkan periode yang sama tahun sebelumnya. Perangkat IoT sering kali menjadi target empuk karena keterbatasan sumber daya komputasi, minimnya mekanisme keamanan bawaan, serta penggunaan kredensial default yang tidak pernah diubah oleh pengguna akhir.

Metode konvensional untuk mendeteksi serangan—seperti SIEM (*Security Information and Event Management*) dan IDS (*Intrusion Detection System*) berbasis jaringan, menghadapi tantangan kritis ketika infrastruktur jaringan IP utama dikompromikan atau dinonaktifkan oleh penyerang. Dalam skenario *advanced persistent threat* (APT), penyerang sering kali sengaja mematikan jalur logging terlebih dahulu sebelum melancarkan serangan utama, sehingga menghapus jejak evidensi digital.

Proyek **Decentralized IoT Honeypot & Threat Registry** hadir sebagai solusi inovatif yang menggabungkan tiga paradigma teknologi mutakhir:

1. **IoT Honeypot**: Perangkat umpan berbasis ESP32 yang berpura-pura menjadi target yang rentan untuk menarik dan menganalisis aktivitas penyerang.
2. **Out-of-Band Transmission via LoRaWAN**: Penggunaan jaringan Helium (LoRa) sebagai jalur transmisi alternatif yang independen dari infrastruktur internet konvensional.
3. **Immutable Blockchain Registry**: Penyimpanan log ancaman di blockchain

Polygon/Arbitrum menggunakan smart contract, sehingga data tidak dapat dimanipulasi atau dihapus oleh pihak manapun.

1.2 Rumusan Masalah

Permasalahan Utama

1. Bagaimana merancang sistem honeypot IoT berbasis ESP32 yang mampu mendeteksi, mencatat, dan mengklasifikasikan jenis serangan jaringan secara real-time?
2. Bagaimana memanfaatkan protokol LoRaWAN melalui jaringan Helium sebagai jalur transmisi *out-of-band* yang andal untuk mengirimkan log serangan meskipun jalur internet utama terputus?
3. Bagaimana merancang smart contract Solidity pada Ethereum L2 (Polygon/Arbitrum) yang mampu menyimpan *threat log* secara permanen, transparan, dan *tamper-proof*?
4. Bagaimana membangun antarmuka dashboard berbasis Next.js yang terintegrasi dengan MetaMask untuk memvisualisasikan dan memverifikasi data ancaman dari blockchain?

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk:

- Membangun prototipe sistem deteksi ancaman IoT yang resilien terhadap serangan yang mencoba menonaktifkan sistem logging.
- Mengimplementasikan arsitektur *zero-trust logging* di mana data ancaman tidak dapat dipalsukan atau dihapus karena disimpan secara on-chain.
- Menciptakan threat registry terdesentralisasi yang dapat digunakan sebagai sumber intelijen ancaman bersama (*shared threat intelligence*) oleh komunitas keamanan siber.
- Mendemonstrasikan kelayakan teknis integrasi IoT—LoRaWAN—Blockchain sebagai arsitektur keamanan generasi berikutnya.

1.4 Manfaat Penelitian

Table 1.1: Manfaat Penelitian bagi Berbagai Stakeholder

Stakeholder	Manfaat
Akademisi	Kontribusi pada riset integrasi IoT-blockchain untuk keamanan siber
Industri	Blueprint arsitektur untuk sistem monitoring jaringan yang tahan serangan
Regulator	Bukti digital immutable yang dapat digunakan dalam proses hukum dan forensik
Komunitas	Threat registry terdesentralisasi sebagai shared intelligence platform

Chapter 2

Tinjauan Pustaka & Landasan Teori

2.1 IoT Honeypot: Konsep dan Implementasi

2.1.1 Definisi Honeypot

Honeypot adalah sistem keamanan yang dirancang untuk menarik, mendeteksi, dan menganalisis aktivitas penyerang dengan berpura-pura menjadi target yang memiliki kerentanan nyata. Berbeda dengan sistem produksi yang melindungi aset riil, honeypot berfungsi sebagai *decoy* yang memungkinkan tim keamanan untuk mempelajari teknik, taktik, dan prosedur (TTP) penyerang tanpa risiko terhadap infrastruktur aktual.

Menurut taksonomi Lance Spitzner [1], honeypot dapat dikategorikan berdasarkan tingkat interaksi:

Table 2.1: Klasifikasi Honeypot Berdasarkan Tingkat Interaksi

Tipe	Kompleksitas	Karakteristik
Low-Interaction	Rendah	Mensimulasikan layanan terbatas; mudah di-deploy; risiko rendah; data terbatas
Medium-Interaction	Sedang	Mensimulasikan OS partial; lebih banyak data; risiko moderat
High-Interaction	Tinggi	Sistem nyata dengan kerentanan; data kaya; risiko tinggi jika dikompromikan

Proyek ini mengimplementasikan **low-interaction honeypot** pada ESP32, mensimulasikan layanan jaringan umum seperti HTTP (port 80), Telnet (port 23), SSH (port 22), dan MQTT (port 1883) yang sering diserang pada perangkat IoT.

2.1.2 Serangan Umum pada Perangkat IoT

Berdasarkan laporan OWASP IoT Attack Surface Areas [2], vektor serangan utama pada ekosistem IoT meliputi:

- **Port Scanning:** Pemindaian sistematis untuk menemukan layanan yang terbuka menggunakan tools seperti Nmap atau Masscan.
- **Credential Brute-Force:** Percobaan login berulang menggunakan wordlist umum (admin/admin, root/root, dll.) pada Telnet, SSH, dan antarmuka web.
- **Mirai-style Botnet Infection:** Perangkat IoT diinfeksi untuk menjadi bagian dari botnet yang digunakan untuk serangan DDoS.
- **Firmware Exploitation:** Eksploitasi kerentanan pada firmware perangkat untuk mendapatkan akses shell.
- **MQTT Injection:** Penyalahgunaan broker MQTT yang tidak terautentikasi untuk menyisipkan perintah berbahaya.

2.2 ESP32: Platform Mikrokontroler untuk Honeypot

2.2.1 Spesifikasi Teknis

ESP32 merupakan System-on-Chip (SoC) produksi Espressif Systems yang ideal untuk implementasi honeypot IoT karena kombinasi kemampuan jaringan, komputasi, dan harga yang terjangkau.

Table 2.2: Spesifikasi Teknis ESP32

Komponen	Spesifikasi
Prosesor	Dual-core Xtensa LX6, 240 MHz
Memori RAM	520 KB SRAM
Flash	4–16 MB (bergantung modul)
WiFi	802.11 b/g/n, 2.4 GHz
Bluetooth	BT 4.2 + BLE
GPIO	34 pin
ADC/DAC	12-bit ADC (18 ch), 8-bit DAC (2 ch)
Konsumsi Daya	240 mA (aktif), 10 μ A (deep sleep)
Harga	USD 3–8 per unit

2.2.2 Kapabilitas Honeypot pada ESP32

ESP32 mampu menjalankan server TCP/UDP multi-port secara bersamaan menggunakan FreeRTOS *task scheduling*. Setiap port dapat dikonfigurasi untuk:

1. Menerima koneksi masuk dan mencatat IP sumber.
2. Merespons dengan *banner* palsu yang menyerupai layanan nyata untuk mempertahankan keterlibatan penyerang lebih lama.
3. Mengklasifikasikan jenis serangan berdasarkan pola traffic.
4. Mengirimkan alert melalui WiFi (primary) atau LoRa (backup/out-of-band).

2.3 LoRaWAN & Jaringan Helium

2.3.1 LoRa dan LoRaWAN

LoRa (*Long Range*) adalah teknologi modulasi radio *spread-spectrum* yang dikembangkan oleh Semtech Corporation. LoRaWAN adalah protokol MAC (*Medium Access Control*) yang berjalan di atas LoRa, mendefinisikan arsitektur jaringan dan komunikasi antara perangkat end-node, gateway, dan network server.

Karakteristik Teknis LoRaWAN

- **Jangkauan:** 2–5 km (perkotaan), 15–20 km (pedesaan/line-of-sight)
- **Frekuensi:** 915 MHz (Amerika), 868 MHz (Eropa), 923 MHz (Asia)
- **Data Rate:** 0.3–50 kbps
- **Payload Maks:** 242 byte per paket
- **Konsumsi Daya:** Sangat rendah (dapat bertahan bertahun-tahun dengan baterai AA)
- **Keamanan:** AES-128 enkripsi end-to-end

2.3.2 Jaringan Helium (The People's Network)

Helium adalah jaringan LoRaWAN terdesentralisasi berbasis blockchain di mana individu dapat menjadi operator hotspot dan mendapatkan reward berupa token HNT (*Helium Network Token*). Helium menggunakan mekanisme konsensus *Proof of Coverage* (PoC) untuk memverifikasi bahwa hotspot benar-benar menyediakan layanan jaringan.

Table 2.3: Perbandingan Helium vs Jaringan LoRaWAN Konvensional

Aspek	Helium	LoRaWAN Konvensional
Kepemilikan	Terdesentralisasi (komunitas)	Terpusat (operator tunggal)
Biaya	Pay-per-byte (Data Credits)	Subscription bulanan
Cakupan	190+ negara	Terbatas per operator
Ketergantungan	Minimal (desentralisasi)	Tinggi (single provider)
Transparansi	Blockchain-verified	Opaque

2.3.3 Relevansi untuk Sistem Keamanan

Nilai strategis LoRaWAN/Helium dalam konteks keamanan siber adalah kemampuannya sebagai **out-of-band communication channel**. Ketika penyerang berhasil memutus koneksi internet utama, jalur LoRa tetap berfungsi karena:

- Menggunakan frekuensi radio yang berbeda (sub-GHz, bukan WiFi/Ethernet).
- Tidak bergantung pada infrastruktur ISP yang mungkin dikompromikan.
- Jangkauan yang cukup untuk menjangkau gateway Helium terdekat.

2.4 Blockchain & Smart Contract

2.4.1 Mengapa Blockchain untuk Threat Logging?

Penyimpanan log keamanan tradisional memiliki kelemahan fundamental: log tersimpan di server yang dapat dikompromikan, dimanipulasi, atau dihapus oleh penyerang yang memiliki akses administrator. Blockchain menawarkan solusi melalui properti-properiti berikut:

Properti Blockchain untuk Keamanan Data

- **Immutability:** Data yang sudah di-commit tidak dapat diubah tanpa konsensus mayoritas jaringan.
- **Transparency:** Semua transaksi dapat diverifikasi secara publik oleh siapa pun.
- **Decentralization:** Tidak ada single point of failure atau control.
- **Timestamping:** Setiap transaksi memiliki timestamp yang kriptografis diverifikasi.
- **Non-repudiation:** Tidak ada pihak yang dapat menyangkal keberadaan data yang telah di-log.

2.4.2 Ethereum Layer-2: Polygon & Arbitrum

Ethereum mainnet memiliki biaya transaksi (*gas fees*) yang tinggi, menjadikannya tidak praktis untuk logging frekuensi tinggi. Solusi Layer-2 seperti Polygon dan Arbitrum menawarkan skalabilitas dengan biaya rendah sambil tetap mewarisi keamanan dari Ethereum mainnet.

Table 2.4: Perbandingan Ethereum L2 untuk Threat Registry

Parameter	Ethereum	Polygon	Arbitrum
Avg Gas Fee	\$5–50	\$0.001–0.01	\$0.01–0.05
TPS	15	7000+	40,000+
Finality	15 min	2 detik	1 menit
Keamanan	Sangat Tinggi	Tinggi	Sangat Tinggi
EVM Compat.	Ya	Ya	Ya

2.4.3 Solidity: Bahasa Smart Contract

Solidity adalah bahasa pemrograman statically-typed berorientasi kontrak yang dirancang khusus untuk Ethereum Virtual Machine (EVM). Versi 0.8.x yang digunakan dalam proyek ini menghadirkan fitur keamanan bawaan seperti *overflow/underflow protection* dan *custom errors*.

Chapter 3

Tujuan dan Ruang Lingkup Proyek

3.1 Tujuan Utama

3.1.1 Tujuan 1: Implementasi Low-Interaction IoT Honeypot

Membangun sistem honeypot berbasis ESP32 yang mampu:

- Mensimulasikan setidaknya 5 layanan jaringan yang umum diserang (HTTP, Telnet, SSH, MQTT, Modbus).
- Mendeteksi dan mengklasifikasikan jenis serangan secara otomatis berdasarkan pola traffic.
- Beroperasi secara mandiri dengan konsumsi daya yang minimal (<500 mA).
- Mencatat metadata serangan meliputi: IP penyerang, port target, jenis serangan, timestamp, dan payload awal.

3.1.2 Tujuan 2: Out-of-Band Transmission via Helium LoRaWAN

Memastikan kontinuitas logging melalui:

- Integrasi modul LoRa (RFM95W/SX1276) dengan ESP32 untuk transmisi data serial.
- Registrasi perangkat pada Helium Console dengan konfigurasi OTAA (*Over-the-Air Activation*).
- Implementasi mekanisme *failover*: WiFi sebagai jalur primer, LoRa sebagai jalur cadangan.

- Kompresi payload menggunakan Cayenne LPP atau format biner kustom untuk efisiensi bandwidth.

3.1.3 Tujuan 3: Immutable Blockchain Threat Registry

Membangun infrastruktur on-chain yang mencakup:

- Smart contract `ThreatRegistry` yang di-deploy pada Polygon Mumbai testnet (kemudian mainnet).
- Fungsi logging, retrieval, dan filtering data ancaman dengan gas optimization.
- Event emission untuk real-time monitoring melalui Web3 event listeners.
- Mekanisme access control menggunakan pola `Ownable` dari OpenZeppelin.

3.1.4 Tujuan 4: Dashboard Monitoring Terdesentralisasi

Mengembangkan antarmuka pengguna berbasis Next.js yang:

- Terhubung ke smart contract melalui MetaMask dan Ethers.js.
- Menampilkan visualisasi real-time threat data (grafik, tabel, peta IP geolokasi).
- Memungkinkan filtering dan pencarian berdasarkan tipe serangan, rentang waktu, dan IP penyerang.
- Menyediakan fitur ekspor data untuk keperluan forensik dan pelaporan.

3.2 Ruang Lingkup Proyek

3.2.1 Dalam Ruang Lingkup

- Pengembangan firmware ESP32 untuk honeypot multi-port.
- Integrasi hardware ESP32 + modul LoRa RFM95W.
- Konfigurasi perangkat pada Helium Console.
- Pengembangan dan deployment smart contract di Polygon.
- Pembangunan middleware webhook untuk bridging Helium ke blockchain.

- Pengembangan frontend dashboard dengan Next.js.
- Pengujian pada jaringan testnet dan simulasi serangan.

3.2.2 Di Luar Ruang Lingkup

- Analisis malware mendalam dari payload serangan.
- Implementasi high-interaction honeypot dengan OS virtualisasi penuh.
- Deployment skala penuh pada infrastruktur produksi.
- Mekanisme respon otomatis atau *active countermeasure*.

Chapter 4

Arsitektur Sistem & Alur Data

4.1 Arsitektur End-to-End

Sistem ini terdiri dari empat lapisan utama yang bekerja secara terintegrasi:

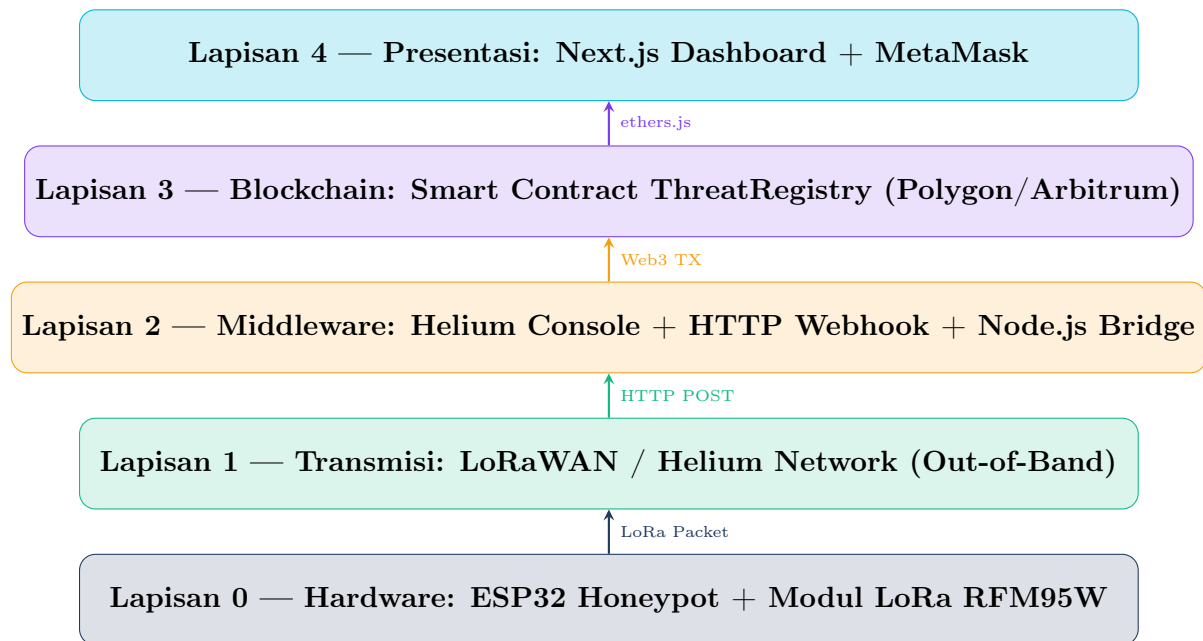


Figure 4.1: Arsitektur Berlapis Sistem Decentralized IoT Honeypot

4.2 Alur Data Detail

4.2.1 Fase 1: Deteksi Serangan (ESP32)

Ketika penyerang melakukan koneksi ke salah satu port yang di-*listen* oleh honeypot:

1. Server TCP pada ESP32 menerima koneksi masuk.
2. Sistem mencatat: IP sumber, port tujuan, timestamp (NTP-synced), dan byte pertama payload.
3. Algoritma klasifikasi sederhana menentukan tipe serangan berdasarkan port dan payload pattern.
4. Data dikemas dalam struktur JSON kompak dan antri untuk transmisi.

4.2.2 Fase 2: Transmisi Out-of-Band (LoRa)

1. ESP32 mencoba mengirim data melalui WiFi ke endpoint cloud (jalur primer).
2. Jika WiFi tidak tersedia atau gagal dalam 5 detik, sistem beralih ke jalur LoRa.
3. Data dikompresi ke format biner (maks 51 byte untuk SF12) dan dikirim via RFM95W.
4. Gateway Helium menerima paket dan meneruskannya ke Helium Network Server.
5. Helium Console memproses paket dan mengirimkan payload ke webhook URL yang telah dikonfigurasi.

4.2.3 Fase 3: Middleware Processing

1. Node.js server menerima HTTP POST dari Helium webhook.
2. Payload di-decode dari Base64 dan di-parse ke struktur data JavaScript.
3. Validasi dan sanitasi data dilakukan sebelum on-chain submission.
4. Middleware memanggil fungsi `logThreat()` pada smart contract via Ethers.js.
5. Transaksi ditandatangani menggunakan private key operator (disimpan sebagai environment variable).

4.2.4 Fase 4: On-Chain Storage (Polygon)

1. Smart contract menerima transaksi dan menyimpan `ThreatLog` ke array storage.
2. Event `ThreatLogged` di-emit untuk notifikasi real-time ke subscriber.
3. Data tersimpan permanen dan tidak dapat diubah pada blockchain Polygon.

4. Block explorer (Polygonscan) mencatat semua transaksi untuk audit publik.

Chapter 5

Tools dan Teknologi yang Digunakan

5.1 Hardware Layer

5.1.1 ESP32 Development Board

ESP32 dipilih sebagai platform honeypot utama karena beberapa alasan teknis dan ekonomis. Dari sisi teknis, kemampuan dual-core memungkinkan pemisahan tugas antara penanganan koneksi jaringan (core 0) dan pemrosesan data/komunikasi LoRa (core 1). Fitur deep sleep dengan konsumsi $<10 \mu A$ memungkinkan deployment jangka panjang dengan sumber daya baterai.

Dari sisi ekonomis, harga ESP32 yang berkisar USD 3–8 per unit membuatnya ideal untuk deployment skala besar sebagai jaringan honeypot terdistribusi, di mana puluhan unit dapat di-deploy di berbagai segmen jaringan dengan biaya yang terjangkau.

5.1.2 Modul LoRa: RFM95W / SX1276

Modul RFM95W berbasis chip SX1276 dari Semtech adalah solusi LoRa yang umum digunakan karena ketersediaan, dokumentasi yang baik, dan kompatibilitas dengan berbagai library. Spesifikasi kunci:

5.2 Network & Connectivity Layer

5.2.1 LoRaWAN Protocol Stack

LoRaWAN mendefinisikan tiga kelas perangkat end-node:

Table 5.1: Spesifikasi Modul RFM95W

Parameter	Nilai
Frekuensi Operasi	868/915 MHz
Output Power Maks	+20 dBm
Sensitivitas RX	-148 dBm (SF12, 125 kHz)
Antarmuka	SPI
Supply Voltage	3.3V
Konsumsi RX	10.3 mA
Konsumsi TX	120 mA (20 dBm)

- **Class A** (digunakan dalam proyek ini): Perangkat menginisiasi uplink, lalu membuka dua jendela downlink singkat. Paling hemat energi.
- **Class B**: Menambahkan jendela downlink terjadwal yang disinkronkan dengan beacon gateway.
- **Class C**: Selalu dalam mode terima kecuali saat transmisi. Konsumsi daya tertinggi.

Honeypot ESP32 dikonfigurasi sebagai **Class A** karena sifat operasinya yang uplink-dominant (mengirim log) dengan kebutuhan downlink yang minimal.

5.2.2 Helium Console & LongFi Protocol

Helium Console adalah antarmuka manajemen untuk perangkat LoRaWAN pada jaringan Helium. Fitur-fitur yang dimanfaatkan:

- **Device Registration**: OTAA join dengan AppEUI, DevEUI, dan AppKey.
- **Decoder Functions**: JavaScript decoder untuk parsing payload biner.
- **Integrations/Webhooks**: HTTP endpoint untuk forwarding data ke middle-ware.
- **Debug Console**: Real-time packet inspection untuk troubleshooting.
- **Data Credits**: Unit pembayaran untuk biaya transmisi data (1 DC = \$0.00001).

LongFi adalah protokol yang dikembangkan Helium yang menggabungkan LoRaWAN dengan Helium Blockchain untuk verifikasi *Proof of Coverage* secara kriptografis.

5.3 Blockchain Layer

5.3.1 Solidity

Solidity adalah bahasa pemrograman berorientasi kontrak yang dieksekusi di atas Ethereum Virtual Machine (EVM). Fitur-fitur Solidity yang relevan untuk proyek ini:

- **Struct:** Mendefinisikan tipe data kustom `ThreatLog`.
- **Dynamic Arrays:** Menyimpan koleksi log yang terus bertambah.
- **Events:** Notifikasi asinkron untuk frontend listener.
- **View Functions:** Pembacaan data on-chain tanpa biaya gas.
- **Access Modifiers:** Kontrol akses berbasis modifier.

5.3.2 Polygon & Arbitrum (Ethereum L2)

Kedua jaringan Layer-2 ini menggunakan pendekatan yang berbeda untuk skalabilitas:

- **Polygon:** Menggunakan sidechain dengan mekanisme *Proof of Stake* dan jembatan (*bridge*) ke Ethereum mainnet.
- **Arbitrum:** Menggunakan *Optimistic Rollup* yang mengasumsikan validitas transaksi dan menggunakan mekanisme *fraud proof* untuk keamanan.

Rekomendasi proyek adalah menggunakan **Polygon** untuk deployment awal karena ekosistem yang lebih matang dan dokumentasi yang lebih lengkap.

5.3.3 Hardhat & Remix IDE

Hardhat adalah development environment profesional untuk Ethereum yang menyediakan:

- Local blockchain (Hardhat Network) untuk testing.
- Framework testing dengan Chai dan Mocha.
- Plugin untuk verifikasi kontrak di Etherscan/Polygonscan.
- Task runner untuk deployment scripts.

Remix IDE berguna untuk prototyping cepat dan debugging smart contract langsung di browser.

5.4 Middleware & Web3 Layer

5.4.1 Helium Console Webhooks

Webhook pada Helium Console dikonfigurasi untuk mengirim HTTP POST request setiap kali paket data diterima dari perangkat. Format payload:

Listing 5.1: Format Payload Helium Webhook

```
1 {  
2   "name": "esp32-honeypot-01",  
3   "dev_eui": "XXXXXXXXXXXX",  
4   "payload": "<base64-encoded-data>",  
5   "metadata": {  
6     "frequency": 915.2,  
7     "datarate": "SF10BW125",  
8     "snr": 9.8,  
9     "rssi": -98  
10  },  
11   "reported_at": 1700000000000  
12 }
```

5.4.2 Ethers.js

Ethers.js adalah library JavaScript untuk interaksi dengan jaringan Ethereum-compatible. Library ini dipilih karena ukurannya yang lebih kecil dibandingkan Web3.js, TypeScript support yang baik, dan API yang lebih modern dan mudah dipahami.

5.5 Frontend Layer

5.5.1 Next.js

Next.js adalah React framework yang menyediakan:

- Server-Side Rendering (SSR) untuk performa optimal.
- API Routes untuk backend-in-frontend architecture.

- Static Site Generation (SSG) untuk halaman yang tidak memerlukan data dinamis.
- Built-in optimizations (code splitting, image optimization).

5.5.2 MetaMask Integration

MetaMask berfungsi sebagai wallet browser yang memungkinkan pengguna untuk:

- Mengautentikasi ke dashboard menggunakan Ethereum address.
- Menandatangani transaksi untuk submit threat report manual.
- Berinteraksi dengan smart contract secara langsung dari browser.

Chapter 6

Implementasi Teknis

6.1 Smart Contract: ThreatRegistry

6.1.1 Kode Kontrak Dasar

Berikut adalah implementasi smart contract `ThreatRegistry` dalam Solidity:

Listing 6.1: `ThreatRegistry.sol` — Versi Dasar

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract ThreatRegistry {
5     struct ThreatLog {
6         uint256 timestamp;
7         string attackerIP;
8         string attackType;
9     }
10
11     ThreatLog[] public threatLogs;
12
13     event ThreatLogged(
14         uint256 timestamp,
15         string attackerIP,
16         string attackType
17     );
18
19     function logThreat(
20         string memory _attackerIP,
21         string memory _attackType
22     ) public {
23         threatLogs.push(ThreatLog({
```

```
24         timestamp: block.timestamp,
25         attackerIP: _attackerIP,
26         attackType: _attackType
27     }));
28
29     emit ThreatLogged(
30         block.timestamp,
31         _attackerIP,
32         _attackType
33     );
34 }
35
36 function getThreatLogs()
37     public
38     view
39     returns (ThreatLog[] memory)
40 {
41     return threatLogs;
42 }
43 }
```

6.1.2 Versi Diperkuat dengan Access Control

Versi produksi mengintegrasikan Ownable dari OpenZeppelin untuk membatasi siapa yang dapat menulis ke registry:

Listing 6.2: ThreatRegistry.sol — Versi Produksi dengan Access Control

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 import "@openzeppelin/contracts/access/Ownable.sol";
5 import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
6
7 contract ThreatRegistry is Ownable, ReentrancyGuard {
8     struct ThreatLog {
9         uint256 id;
10        uint256 timestamp;
11        string attackerIP;
12        string attackType;
13        string sourceDevice;
14        uint8 severityLevel; // 1=Low, 2=Med, 3=High, 4=Crit
15    }
16
17    ThreatLog[] private threatLogs;
18    uint256 private logCounter;
```

```

19
20 // Authorized reporters (honeypot devices)
21 mapping(address => bool) public authorizedReporters;
22
23 event ThreatLogged(
24     uint256 indexed id,
25     uint256 timestamp,
26     string attackerIP,
27     string attackType,
28     uint8 severityLevel
29 );
30 event ReporterAuthorized(address reporter);
31 event ReporterRevoked(address reporter);
32
33 modifier onlyAuthorized() {
34     require(
35         authorizedReporters[msg.sender] || msg.sender == owner
36         (),
37         "Not authorized to log threats"
38     );
39 }
40
41 function authorizeReporter(address _reporter)
42     external onlyOwner
43 {
44     authorizedReporters[_reporter] = true;
45     emit ReporterAuthorized(_reporter);
46 }
47
48 function logThreat(
49     string memory _attackerIP,
50     string memory _attackType,
51     string memory _sourceDevice,
52     uint8 _severityLevel
53 ) external onlyAuthorized nonReentrant {
54     require(_severityLevel >= 1 && _severityLevel <= 4,
55         "Invalid severity level");
56
57     logCounter++;
58     threatLogs.push(ThreatLog({
59         id: logCounter,
60         timestamp: block.timestamp,
61         attackerIP: _attackerIP,
62         attackType: _attackType,
63         sourceDevice: _sourceDevice,
64         severityLevel: _severityLevel

```

```
65     });
66
67     emit ThreatLogged(
68         logCounter, block.timestamp,
69         _attackerIP, _attackType, _severityLevel
70     );
71 }
72
73 function getThreatLogs()
74     external view returns (ThreatLog[] memory)
75 {
76     return threatLogs;
77 }
78
79 function getThreatCount()
80     external view returns (uint256)
81 {
82     return logCounter;
83 }
84 }
```

6.2 Firmware ESP32 Honeypot

6.2.1 Struktur Utama Firmware

Listing 6.3: ESP32 Honeypot — Core Logic (Arduino/ESP-IDF)

```
1 #include <Arduino.h>
2 #include <WiFi.h>
3 #include <SPI.h>
4 #include <LoRa.h>
5 #include <ArduinoJson.h>
6 #include <time.h>
7
8 // LoRa Pin Configuration (RFM95W)
9 #define LORA_SCK    18
10 #define LORA_MISO   19
11 #define LORA_MOSI   23
12 #define LORA_SS     5
13 #define LORA_RST    14
14 #define LORA_DIO0    26
15 #define LORA_BAND    915E6
16
17 // Honeypot Ports to Monitor
```

```

18 const int HONEYPOT_PORTS[] = {22, 23, 80, 1883, 502};
19 const char* PORT_SERVICES[] = {
20     "SSH", "Telnet", "HTTP", "MQTT", "Modbus"
21 };
22 const int NUM_PORTS = 5;
23
24 WiFiServer* servers[5];
25
26 struct AttackLog {
27     String attackerIP;
28     int targetPort;
29     String serviceType;
30     String attackType;
31     long timestamp;
32 };
33
34 void classifyAttack(AttackLog& log, WiFiClient& client) {
35     String payload = "";
36     if (client.available()) {
37         payload = client.readStringUntil('\n');
38         payload.trim();
39     }
40
41     if (log.targetPort == 22) {
42         log.attackType = "SSH_BRUTEFORCE_ATTEMPT";
43     } else if (log.targetPort == 23) {
44         log.attackType = payload.length() > 0
45             ? "TELNET_CREDENTIAL_ATTEMPT" : "TELNET_PORT_SCAN";
46     } else if (log.targetPort == 80) {
47         log.attackType = payload.startsWith("GET")
48             ? "HTTP_RECONNAISSANCE" : "HTTP_PORT_SCAN";
49     } else if (log.targetPort == 1883) {
50         log.attackType = "MQTT_UNAUTHORIZED_ACCESS";
51     } else {
52         log.attackType = "PORT_SCAN";
53     }
54 }
55
56 void transmitViaLoRa(const AttackLog& log) {
57     // Pack into compact binary format
58     // [1B type][4B timestamp][4B IP octets][1B port_idx]
59     uint8_t packet[10];
60     IPAddress ip;
61     ip.fromString(log.attackerIP);
62
63     packet[0] = 0x01; // Threat log marker
64     packet[1] = (log.timestamp >> 24) & 0xFF;

```

```

65   packet[2] = (log.timestamp >> 16) & 0xFF;
66   packet[3] = (log.timestamp >> 8) & 0xFF;
67   packet[4] = (log.timestamp      ) & 0xFF;
68   packet[5] = ip[0]; packet[6] = ip[1];
69   packet[7] = ip[2]; packet[8] = ip[3];
70   packet[9] = (uint8_t)log.targetPort / 100;
71
72   LoRa.beginPacket();
73   LoRa.write(packet, 10);
74   LoRa.endPacket();
75 }
76
77 void setup() {
78   Serial.begin(115200);
79   WiFi.begin(WIFI_SSID, WIFI_PASS);
80
81   // Initialize LoRa
82   LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
83   LoRa.begin(LORA_BAND);
84   LoRa.setSpreadingFactor(10);
85
86   // Start honeypot servers
87   for (int i = 0; i < NUM_PORTS; i++) {
88     servers[i] = new WiFiServer(HONEYPOT_PORTS[i]);
89     servers[i]->begin();
90   }
91 }
92
93 void loop() {
94   for (int i = 0; i < NUM_PORTS; i++) {
95     WiFiClient client = servers[i]->available();
96     if (client) {
97       AttackLog log;
98       log.attackerIP = client.remoteIP().toString();
99       log.targetPort = HONEYPOT_PORTS[i];
100      log.serviceType = PORT_SERVICES[i];
101      log.timestamp = time(nullptr);
102
103      classifyAttack(log, client);
104      client.stop();
105
106      // Try WiFi first, fallback to LoRa
107      if (WiFi.status() == WL_CONNECTED) {
108        sendViaHTTP(log);
109      } else {
110        transmitViaLoRa(log);
111      }

```

```
112     }  
113   }  
114 }
```

6.3 Middleware: Node.js Webhook Bridge

Listing 6.4: Helium-to-Blockchain Bridge (Node.js + Ethers.js)

```
1  const express = require('express');  
2  const { ethers } = require('ethers');  
3  const app = express();  
4  app.use(express.json());  
5  
6  const CONTRACT_ABI = [  
7    "function logThreat(string,string,string,uint8) external",  
8    "event ThreatLogged(uint256,uint256,string,string,uint8)"  
9  ];  
10  
11  const provider = new ethers.JsonRpcProvider(  
12    process.env.POLYGON_RPC_URL  
13  );  
14  const wallet = new ethers.Wallet(  
15    process.env.PRIVATE_KEY, provider  
16  );  
17  const contract = new ethers.Contract(  
18    process.env.CONTRACT_ADDRESS, CONTRACT_ABI, wallet  
19  );  
20  
21  // Helium decoder for binary LoRa payload  
22  function decodeHeliumPayload(base64Payload) {  
23    const buf = Buffer.from(base64Payload, 'base64');  
24    const timestamp = buf.readUInt32BE(1);  
25    const ip = `${buf[5]}.${buf[6]}.${buf[7]}.${buf[8]}`;  
26    const portMap = { 0: "PORT_SCAN", 1: "SSH", 2: "TELNET" };  
27    return {  
28      attackerIP: ip,  
29      attackType: portMap[buf[9]] || "UNKNOWN",  
30      sourceDevice: "esp32-honeypot",  
31      severity: 2  
32    };  
33  }  
34  
35  app.post('/helium-webhook', async (req, res) => {  
36    try {  
37      const decoded = decodeHeliumPayload(req.body.payload);
```

```
38
39     const tx = await contract.logThreat(
40         decoded.attackerIP,
41         decoded.attackType,
42         decoded.sourceDevice,
43         decoded.severity
44     );
45     await tx.wait();
46
47     console.log('Threat logged: TX ${tx.hash}');
48     res.json({ success: true, txHash: tx.hash });
49 } catch (err) {
50     console.error('Bridge error:', err);
51     res.status(500).json({ error: err.message });
52 }
53 });
54
55 app.listen(3001, () =>
56     console.log('Helium-Blockchain bridge running on :3001')
57 );
```

Chapter 7

Daftar Referensi

7.1 Referensi Utama

7.1.1 Helium Network Documentation

Helium Network Docs

Judul: Helium Network Documentation

URL: <https://docs.helium.com>

Kategori: Dokumentasi Resmi

Konten Relevan:

- *Device and Sensor Developer* — Panduan registrasi perangkat LoRaWAN dengan OTAA, konfigurasi AppEUI/DevEUI/AppKey.
- *Helium Console Integration* — Setup webhook HTTP, decoder JavaScript, dan monitoring real-time.
- *LongFi Protocol Specification* — Detail teknis protokol komunikasi Helium.
- *Data Credits & Pricing* — Model pembayaran berbasis konsumsi data.
- *Coverage Map API* — API untuk memverifikasi ketersediaan jaringan di lokasi tertentu.

7.1.2 Solidity & Ethereum Documentation

Solidity Language Documentation

Judul: Solidity Documentation v0.8.x

URL: <https://docs.soliditylang.org>

Kategori: Dokumentasi Resmi Bahasa Pemrograman

Konten Relevan:

- *Types* — Tipe data `uint256`, `string`, `address`, `struct`.
- *Contracts* — Definisi kontrak, konstruktor, modifier, dan fungsi.
- *Events and Logs* — Implementasi event emission dan indexing.
- *Security Considerations* — Reentrancy, integer overflow, dan best practices keamanan.
- *ABI Encoding* — Standar encoding untuk interoperabilitas antar kontrak.

7.1.3 Konsep IoT Honeypot (IEEE/Jurnal Akademik)

IEEE IoT Honeypot Research

Sumber Primer:

- Dowling, S., et al. (2017). “*An IoT Honeypot: Detecting and Classifying Attacks Against IoT Devices.*” IEEE International Conference on Communications.
- Kolias, C., et al. (2017). “*DDoS in the IoT: Mirai and Other Botnets.*” IEEE Computer, 50(7), 80–84.
- Pa, Y.M.P., et al. (2015). “*IoT POT: Analysing the Rise of IoT Compromises.*” 9th USENIX Workshop on Offensive Technologies.
- Nawrocki, M., et al. (2016). “*A Survey on Honeypot Software and Data Analysis.*” arXiv:1608.06249.

Database: IEEE Xplore (<https://ieeexplore.ieee.org>)

Relevansi: Dasar teori implementasi low-interaction honeypot pada resource-constrained devices.

7.2 Referensi Tambahan

7.2.1 Ethers.js Documentation

Ethers.js v6 Documentation

Judul: ethers.js Documentation

URL: <https://docs.ethers.org/v6/>

Kategori: Library Documentation

Konten Relevan:

- *Providers* — Koneksi ke jaringan Polygon via RPC endpoint.
- *Signers* — Wallet management dan transaction signing.
- *Contract Interaction* — Memanggil fungsi smart contract dan menangani events.
- *Utils* — Parsing ABI, encoding/decoding data, dan unit conversion (Wei/Ether).
- *Event Filtering* — Real-time listening pada event `ThreatLogged` dari kontrak.

7.2.2 OWASP Internet of Things Project

OWASP IoT Project

Judul: OWASP Internet of Things Project

URL: <https://owasp.org/www-project-internet-of-things/>

Kategori: Security Standard & Best Practices

Konten Relevan:

- *IoT Attack Surface Areas* — Katalog komprehensif vektor serangan pada ekosistem IoT.
- *IoT Top 10 Vulnerabilities* — Sepuluh kerentanan paling kritis pada perangkat IoT (termasuk weak credentials, insecure network services).
- *Firmware Analysis Guide* — Metodologi analisis firmware untuk menemukan kerentanan.
- *Testing Guide* — Panduan pengujian keamanan end-to-end untuk produk IoT.

Relevansi: Panduan dalam mendefinisikan attack surface yang disimulasikan oleh honeypot.

7.2.3 OpenZeppelin Contracts

OpenZeppelin Smart Contract Library

Judul: OpenZeppelin Contracts v5

URL: <https://docs.openzeppelin.com/contracts/5.x/>

GitHub: <https://github.com/OpenZeppelin/openzeppelin-contracts>

Kategori: Audited Smart Contract Library

Komponen yang Digunakan:

- `Ownable.sol` — Manajemen kepemilikan kontrak dan pembatasan akses fungsi sensitif.
- `ReentrancyGuard.sol` — Perlindungan terhadap serangan reentrancy.
- `Pausable.sol` — Mekanisme pause/unpause kontrak dalam kondisi darurat.
- `AccessControl.sol` — Sistem peran (role-based access) yang lebih granular.

Relevansi: Memastikan smart contract memenuhi standar keamanan yang telah diaudit oleh komunitas.

7.2.4 Referensi Tambahan Lainnya

Table 7.1: Daftar Referensi Tambahan

Sumber	Konten	URL
Hardhat Docs	Development environment, testing, deployment scripts	hardhat.org
Polygon Docs	RPC endpoints, bridging, gas management	docs.polygon.technology
Semtech SX1276 Datasheet	Register map, LoRa parameter tuning	semtech.com
Arduino LoRa Library	SPI communication dengan RFM95W	github.com/sandeepmistry
NIST SP 800-82r3	Industrial/IoT network security guidelines	csrc.nist.gov
Wagmi/Viem Docs	Modern React hooks untuk Web3 frontend	wagmi.sh

Chapter 8

Rencana Pengujian & Evaluasi

8.1 Skenario Pengujian

8.1.1 Test Case 1: Deteksi Port Scan

Catatan Etika

Seluruh pengujian dilakukan dalam lingkungan terisolasi (lab network) terhadap perangkat milik tim sendiri. Penggunaan teknik serangan terhadap sistem tanpa izin adalah ilegal.

Prosedur: Jalankan `nmap -sS -p 22,23,80,1883 <honeypot-ip>` dari mesin pengujian.

Expected Result: Log tercatat dalam 5 detik, dikirim via LoRa/WiFi, dan muncul sebagai transaksi blockchain dalam 30 detik.

8.1.2 Test Case 2: Out-of-Band Failover

Prosedur: Putus koneksi WiFi ESP32, kemudian lakukan port scan. Verifikasi bahwa log tetap terkirim via LoRa.

Expected Result: Log berhasil diterima gateway Helium dan diproses ke blockchain dalam 60–120 detik (toleransi latency LoRa).

8.1.3 Test Case 3: Immutability Verification

Prosedur: Coba panggil fungsi modifikasi data melalui akun non-owner pada smart contract.

Expected Result: Transaksi revert dengan error "Not authorized to log threats".

8.2 Metrik Keberhasilan

Table 8.1: Key Performance Indicators (KPI) Proyek

KPI	Deskripsi	Target
Detection Rate	Persentase serangan yang berhasil dideteksi	>95%
False Positive	Log yang salah mengklasifikasi traffic legitim	<5%
LoRa Reliability	Tingkat keberhasilan pengiriman paket LoRa	>90%
Blockchain Latency	Waktu dari serangan hingga on-chain confirmation	<120 detik
Gas Efficiency	Biaya gas per <code>logThreat()</code> transaksi	<0.01 MATIC
Dashboard Uptime	Ketersediaan dashboard monitoring	>99%

Chapter 9

Kesimpulan & Pengembangan Lanjutan

9.1 Kesimpulan

Proyek **Decentralized IoT Honeypot & Threat Registry** mengintegrasikan tiga domain teknologi yang saling melengkapi untuk menciptakan sistem deteksi ancaman yang lebih resilien dibandingkan pendekatan konvensional:

1. **Resiliensi Jaringan:** Penggunaan LoRaWAN sebagai jalur out-of-band memastikan kontinuitas logging bahkan ketika infrastruktur internet utama dikompromikan atau dinonaktifkan.
2. **Integritas Data:** Blockchain Polygon menjamin bahwa setiap log ancaman yang tersimpan tidak dapat dimanipulasi, dihapus, atau disangkal—memberikan bukti digital yang sah secara kriptografis.
3. **Desentralisasi:** Tidak ada single point of failure dalam sistem ini. Data tersebar di ribuan node blockchain, sementara jaringan Helium tersebar di ribuan hotspot komunitas.
4. **Skalabilitas Ekonomis:** Biaya ESP32 yang rendah dan biaya transaksi Polygon yang minimal memungkinkan deployment skala besar sebagai jaringan honeypot terdistribusi.

9.2 Pengembangan Lanjutan

- **Machine Learning Integration:** Implementasi model klasifikasi serangan berbasis ML (TinyML pada ESP32 atau cloud inference) untuk meningkatkan akurasi identifikasi tipe serangan.
- **Multi-Device Mesh:** Pembangunan jaringan honeypot terdistribusi dengan agregasi dan korelasi ancaman lintas-node.
- **DAO Governance:** Transformasi threat registry menjadi platform intelijen ancaman komunitas dengan mekanisme tata kelola DAO (*Decentralized Autonomous Organization*).
- **NFT-based Threat Certificates:** Penerbitan NFT sebagai sertifikat bukti serangan yang dapat digunakan dalam proses hukum digital.
- **Zero-Knowledge Proofs:** Penggunaan ZK-proof untuk memungkinkan verifikasi keberadaan threat log tanpa mengungkapkan informasi sensitif (IP penyerang) secara publik.

Bibliography

- [1] Spitzner, L. (2002). *Honeypots: Tracking Hackers*. Addison-Wesley Professional.
- [2] OWASP Foundation. (2023). *OWASP Internet of Things Project: IoT Attack Surface Areas*. <https://owasp.org/www-project-internet-of-things/>
- [3] Helium Systems Inc. (2023). *Helium Network Documentation*. <https://docs.helium.com>
- [4] Ethereum Foundation. (2023). *Solidity Documentation v0.8.x*. <https://docs.soliditylang.org>
- [5] Dowling, S., Schukat, M., & Melvin, H. (2017). A ZigBee Honeypot to Assess IoT Cyberattack Behaviour. *IEEE 28th Irish Signals and Systems Conference (ISSC)*, 1–6.
- [6] Kolias, C., Kambourakis, G., Stavrou, A., & Voas, J. (2017). DDoS in the IoT: Mirai and Other Botnets. *IEEE Computer*, 50(7), 80–84.
- [7] Pa, Y. M. P., Suzuki, S., Yoshioka, K., Matsumoto, T., Kasama, T., & Rossow, C. (2015). IoTPOT: Analysing the Rise of IoT Compromises. *9th USENIX Workshop on Offensive Technologies (WOOT)*.
- [8] ethers.js contributors. (2023). *ethers.js Documentation v6*. <https://docs.ethers.org/v6/>
- [9] OpenZeppelin. (2023). *OpenZeppelin Contracts v5.0: Secure Smart Contract Library*. <https://docs.openzeppelin.com/contracts/5.x/>
- [10] Polygon Technology. (2023). *Polygon Documentation*. <https://docs.polygon.technology>
- [11] NIST. (2023). *SP 800-213A: IoT Device Cybersecurity Guidance for the Federal Government*. <https://csrc.nist.gov>

- [12] Semtech Corporation. (2020). *SX1276/77/78/79 Datasheet: 137 MHz to 1020 MHz Low Power Long Range Transceiver*. Semtech.